8-Puzzle Game Solver: Implementation and Analysis

Introduction

The 8-puzzle is a classic sliding puzzle that consists of a 3×3 grid with 8 numbered tiles and one blank space. The objective is to rearrange the tiles from an initial configuration to a goal configuration by sliding tiles into the blank space. This document explains the implementation of an 8-puzzle solver using the A* search algorithm with two different heuristic functions.

Show Image

Key Components of the Implementation

1. Node Class

The `Node` class represents a state in the puzzle, containing:

- The current state (3×3 grid configuration)
- The action that led to this state ('L', 'R', 'U', 'D')
- A reference to the parent node
- The path cost (number of moves from the initial state)

Important methods include:

- `get_successors()`: Generates all possible next states
- `get_successor(action, state)`: Creates a new state after applying a specific action
- `get_blank_pos(state)`: Finds the position of the blank tile

python Copy

```python
class Node:
  def __init__(self, state, action=None, parent=None, path_cost=0):
    self.state = state  # 2D list (3x3)
    self.id = str(self)  # identifier of node
    self.action = action
    self.parent = parent
    self.path_cost = path_cost  # g(n) in A*
```

2. Special Rule Implementation

This implementation includes a unique twist on the traditional 8-puzzle:

- After moving the blank tile, the implementation checks if tiles 1 and 3, or 2 and 4 are adjacent
- If they are adjacent, they automatically swap positions
- This adds complexity to the puzzle and makes it more challenging to solve

python Copy

```python
# Special Rule: Swap (1 <-> 3) and (2 <-> 4) if they are adjacent **after** the blank move
swap_pairs = {(1, 3), (3, 1), (2, 4), (4, 2)}

# Check horizontally adjacent values
for i in range(3):
  for j in range(2):
    if (state[i][j], state[i][j+1]) in swap_pairs:
      state[i][j], state[i][j+1] = state[i][j+1], state[i][j]

# Check vertically adjacent values
for i in range(2):
  for j in range(3):
    if (state[i][j], state[i+1][j]) in swap_pairs:
      state[i][j], state[i+1][j] = state[i+1][j], state[i][j]
```

3. Multiple Goal States

Unlike traditional 8-puzzle implementations that have a single goal state, this version has four possible goal states:

1. Classic goal state with blank in bottom right:

Copy

```
1 2 3
4 5 6
7 8 _
```

2. Reverse ordered goal state with blank in bottom right:

Copy

```
8 7 6
5 4 3
2 1 _
```

3. Classic goal state with blank in top left:

Copy

```
_ 1 2
3 4 5
6 7 8
```

4. Reverse ordered goal state with blank in top right:

Copy

```
8 7 _
6 5 4
3 2 1
```

The solver accepts any of these states as a valid solution.

4. Heuristic Functions

Two heuristic functions are implemented for the A* search algorithm:

Manhattan Distance

- Calculates the sum of horizontal and vertical distances each tile must move to reach its goal position
- Admissible and consistent, ensuring optimality of solutions
- Generally provides better performance than misplaced tiles

python Copy

```python
def manhattan_distance(state, goal_state):
  distance = 0
  for i in range(3):
    for j in range(3):
      if state[i][j] != 0:  # Skip the blank tile
        for gi in range(3):
          for gj in range(3):
            if goal_state[gi][gj] == state[i][j]:
              distance += abs(i - gi) + abs(j - gj)
              break
  return distance
```

Misplaced Tiles

- Counts the number of tiles that are not in their correct position
- Simpler but typically less informative than Manhattan distance
- Still admissible, but may expand more nodes

python Copy

```python
def misplaced_tiles(state, goal_state):
  count = 0
  for i in range(3):
    for j in range(3):
      if state[i][j] != 0 and state[i][j] != goal_state[i][j]:
        count += 1
  return count
```

5. A* Search Algorithm

The A* search algorithm finds the optimal path from the initial state to a goal state:

- Uses a priority queue to select nodes based on f(n) = g(n) + h(n)
  - g(n): Path cost from the start node to the current node
  - h(n): Estimated cost from the current node to the goal
- Maintains a closed set to avoid revisiting states
- Uses the minimum heuristic value across all goal states to ensure admissibility

Key steps in the algorithm:

1. Initialize the frontier with the start node
2. Loop until the frontier is empty or max nodes limit is reached:
   - Remove the node with the lowest f-value
   - Check if it's a goal state; if so, return the solution path
   - Add the node to the closed set
   - Generate successors and add them to the frontier if they're not in the closed set or have a lower path cost

python Copy

```python
def a_star_search(initial_state, heuristic_func, max_nodes=10000):
  start_node = Node(initial_state)

  # Priority queue for frontier (f=g+h)
  frontier = []
  frontier_dict = {}

  # Use the node ID (string representation) for the closed set
  closed_set = set()

  # The f(n) value is the sum of g(n) (path cost) and h(n) (heuristic)
  h_value = get_min_heuristic(start_node.state, heuristic_func)
  f_value = start_node.path_cost + h_value

  # Push (f_value, node) to the priority queue
  entry_count = 0
  heapq.heappush(frontier, (f_value, entry_count, start_node))
  frontier_dict[start_node.id] = (f_value, start_node.path_cost)
  entry_count += 1

  nodes_expanded = 0
  visited_nodes = [start_node]  # For visualization

  while frontier and nodes_expanded < max_nodes:
    # Main A* algorithm loop...
```

6. Visualization

The implementation includes visualization tools using `graphviz`:

- `draw_tree()`: Creates a directed graph of the search tree
- `save_and_display_search_graphs()`: Saves and displays both the full search tree and the solution path

Example visualization of a solution path:

Show Image

7. Random Puzzle Generation

The `generate_random_puzzle()` function creates random but solvable puzzle configurations:

1. Starts with a goal state
2. Applies random valid moves to shuffle the puzzle
3. Returns the resulting state as the initial puzzle configuration

python Copy

```python
def generate_random_puzzle(max_steps = 100):
    """Generates a random solvable 8-puzzle state."""
    # Start with the first goal state
    state = deepcopy(GOAL_STATES[0])
    node = Node(state)

    for _ in range(max_steps):
        successors = node.get_successors()
        if successors:
            node = random.choice(successors)

    return node.state
```

Algorithmic Analysis

Time Complexity

- Worst-case time complexity of A* search: O(b^d), where:
  - b is the branching factor (maximum 4 for the 8-puzzle)
  - d is the solution depth
- With good heuristics like Manhattan distance, the average case is much better

Space Complexity

- O(b^d) as all generated nodes must be stored

Heuristic Comparison

- Manhattan distance typically expands fewer nodes than misplaced tiles
- Both are admissible, ensuring optimal solutions
- Manhattan distance is more informed and thus more efficient

Comparison between heuristics:

Heuristic	Advantages	Disadvantages
Manhattan Distance	More informed, expands fewer nodes	Slightly more complex to compute
Misplaced Tiles	Simpler to implement and understand	Less informed, may expand more nodes

Usage Guide

To use the 8-puzzle solver:

1. Run `solve_8puzzle()` function
2. Choose a heuristic:
   - 1 for Manhattan Distance
   - 2 for Misplaced Tiles
3. The function will:
   - Generate a random puzzle
   - Solve it using A* search
   - Return the solution path, nodes expanded, and visited nodes
4. Optionally visualize the search tree and solution path

Example usage:

python Copy

```python
solution_path, nodes_expanded, visited_nodes = solve_8puzzle()
if solution_path:
  save_and_display_search_graphs(visited_nodes, solution_path, nodes_expanded,
                                 display_images=True,
                                 path_only=True,
                                 max_display=1000)
```

Conclusion

This implementation provides an efficient solution to the 8-puzzle problem using A* search with admissible heuristics. The special swapping rule adds a unique twist to the traditional puzzle, making it more challenging and interesting. The visualization tools help in understanding the search process and solution path.

The solution demonstrates key concepts in artificial intelligence such as heuristic search, admissibility, and graph visualization, making it a valuable educational tool for understanding pathfinding algorithms.

Appendix A: Full Code Implementation

python Copy

```python
# Full code implementation can be found in the original file
```

Appendix B: References

1. Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
2. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107.